

CSS Fundamentals

Cascading Style Sheets — From Zero to Styled

Module 2 · 40 minutes



Today's Agenda

#	Topic	Time
1	What is CSS?	3 min
2	Selectors	7 min
3	Box Model	6 min
4	Colours & Typography	4 min
5	Layout: Flexbox	6 min
6	Layout: CSS Grid	8 min
7	Responsive Design	4 min
8	Live Examples + Q&A	5 min

Goal: By the end, you'll be able to style a real webpage from scratch.



What is CSS?

CSS (Cascading Style Sheets) controls the **visual presentation** of HTML elements.

```
/* This is CSS */  
h1 {  
  color: red;  
  font-size: 48px;  
}
```

Three ways to apply CSS:

- **Inline** — `<p style="color: red">` ❌ Not recommended
- **Internal** — `<style>` tag in `<head>` 🟡 OK for demos
- **External** — `<link rel="stylesheet" href="style.css">` ✅ Best practice



Why Separate CSS from HTML?

```
<!-- HTML – Structure only -->
<article>
  <h1>Welcome to Cardiff</h1>
  <p>The capital city of Wales.</p>
</article>
```

```
/* CSS – Style only */
article {
  background: #f0f4ff;
  padding: 20px;
  border-radius: 8px;
}

h1 {
  color: #c8102e;
}

p {
  font-size: 18px;
  line-height: 1.6;
}
```

CSS Selectors

```
/* Element selector – applies to ALL <p> tags */
p {
  color: #333;
}

/* Class selector – applies to class="card" */
.card {
  background: white;
  border-radius: 8px;
}

/* ID selector – applies to id="header" (unique!) */
#header {
  background: #c8102e;
}

/* Descendant – <a> inside .nav only */
.nav a {
  color: white;
  text-decoration: none;
}

/* Pseudo-class – on hover */
button:hover {
  background: #007a33;
  cursor: pointer;
}
```



Specificity — Who Wins?

When rules conflict, **specificity** decides the winner:

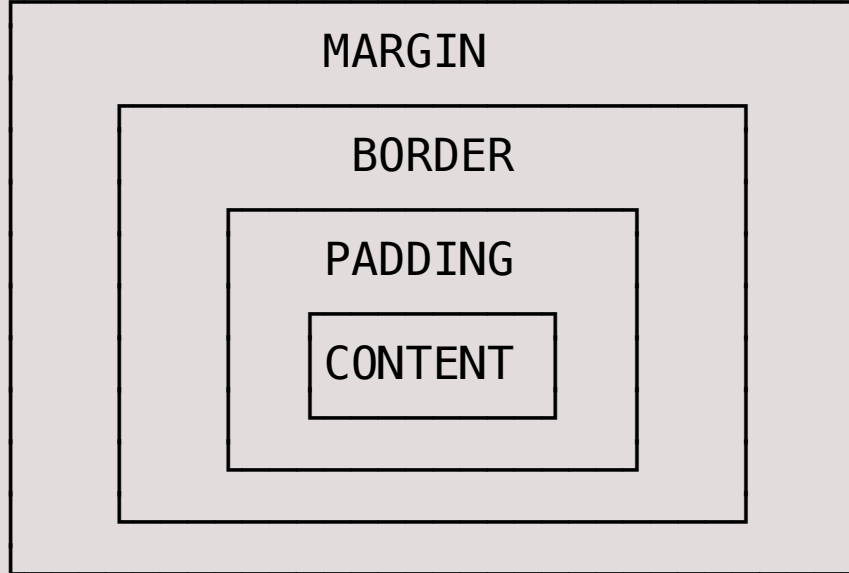
Selector	Points	Example
Inline style	1000	<code>style="color: red"</code>
ID	100	<code>#header</code>
Class / Pseudo	10	<code>.card</code> , <code>:hover</code>
Element	1	<code>p</code> , <code>h1</code>

```
p {  
  color: black;  
} /* 1 point */  
.text {  
  color: green;  
} /* 10 points – wins! */  
#intro {  
  color: red;  
} /* 100 points – wins over both! */
```



The CSS Box Model

Every element is a **rectangle** with four layers:



Tip: Use `box-sizing: border-box` — padding is included in width!



Box Model in Code

```
/* Classic problem */  
.box {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid red;  
  /* Actual width = 300 + 40 + 4 = 344px! 🤯 */  
}
```

```
/* The fix – modern approach */  
*,  
*::before,  
*::after {  
  box-sizing: border-box;  
}
```

```
.box {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid red;  
  /* Actual width = 300px ✅ */  
}
```




Colours & Typography

```
/* Colour formats */
color: red; /* Named */
color: #c8102e; /* Hex */
color: rgb(200, 16, 46); /* RGB */
color: hsl(349, 86%, 42%); /* HSL – most intuitive! */
color: rgba(200, 16, 46, 0.8); /* With opacity */

/* Typography */
font-family: "Segoe UI", Arial, sans-serif;
font-size: 18px; /* or rem – better for accessibility */
font-weight: bold; /* 100–900 */
line-height: 1.6; /* No units – relative to font-size */
text-align: center;
letter-spacing: 0.05em;
```



CSS Variables (Custom Properties)

Define once, use everywhere — like variables in code!

```
/* Define in :root (global scope) */
:root {
  --primary: #c8102e; /* Cardiff Red */
  --success: #007a33; /* Cardiff Green */
  --white: #ffffff;
  --font-base: "Segoe UI", Arial, sans-serif;
  --radius: 8px;
}

/* Use anywhere */
.button {
  background: var(--primary);
  border-radius: var(--radius);
  font-family: var(--font-base);
}

.button:hover {
  background: var(--success); /* Change in one place! */
}
```



Flexbox — Modern Layout

```
/* Parent = flex container */
.navbar {
  display: flex;
  justify-content: space-between; /* horizontal alignment */
  align-items: center; /* vertical alignment */
  gap: 16px;
}

/* Children = flex items */
.nav-item {
  flex: 1; /* grow equally */
}

.logo {
  flex-shrink: 0; /* never shrink */
  width: 120px;
}
```



Flexbox — Alignment Properties

```
.container {  
  display: flex;  
  
  /* Main axis (default: horizontal) */  
  justify-content: flex-start; /* ← default */  
  justify-content: flex-end; /* → */  
  justify-content: center; /* ↔ center */  
  justify-content: space-between; /* |item item| */  
  justify-content: space-around; /* gaps all around */  
  
  /* Cross axis (default: vertical) */  
  align-items: stretch; /* fill height – default */  
  align-items: center; /* vertical center */  
  align-items: flex-end; /* stick to bottom */  
  
  /* Wrapping */  
  flex-wrap: wrap; /* items wrap to next line */  
}
```



CSS Grid — 2D Layout

Flexbox = **one dimension** (row OR column)

Grid = **two dimensions** (rows AND columns simultaneously)

```
.grid {  
  display: grid;  
  
  /* 3 equal columns */  
  grid-template-columns: repeat(3, 1fr);  
  
  /* Mixed: fixed sidebar + flexible main */  
  grid-template-columns: 250px 1fr;  
  
  /* Auto rows */  
  grid-template-rows: auto 1fr auto;  
  
  gap: 20px; /* gutter between all cells */  
}
```

`1fr` = **fraction** of available space — `1fr 2fr 1fr` splits 25% / 50% / 25%



CSS Grid — Placing Items

```
.layout {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: 80px 1fr 60px;  
  gap: 16px;  
}
```

```
/* Line numbers: start / end */
```

```
.header {  
  grid-column: 1 / 4;  
} /* spans all 3 cols */  
.sidebar {  
  grid-column: 1 / 2;  
  grid-row: 2 / 3;  
}
```

```
.main {  
  grid-column: 2 / 4;  
  grid-row: 2 / 3;  
}
```

```
.footer {  
  grid-column: 1 / 4;  
}
```

```
/* Shorthand with span keyword */
```

```
.featured {  
  grid-column: span 2;  
} /* take 2 cols */  
.tall {  
  grid-row: span 3;  
} /* take 3 rows */
```



CSS Grid — Template Areas

Name your zones — the most readable layout approach!

```
.page {  
  display: grid;  
  grid-template-columns: 240px 1fr;  
  grid-template-rows: 70px 1fr 50px;  
  grid-template-areas:  
    "header header"  
    "sidebar main "  
    "footer footer";  
  gap: 16px;  
  min-height: 100vh;  
}  
  
.page-header {  
  grid-area: header;  
  background: var(--color-red);  
}  
  
.page-sidebar {  
  grid-area: sidebar;  
  background: #f5f5f5;  
}  
  
.page-main {  
  grid-area: main;  
}  
  
.page-footer {  
  grid-area: footer;  
  background: #333;
```



Responsive Design

```
/* Base styles – Mobile First */
.card-grid {
  display: flex;
  flex-direction: column;
  gap: 16px;
  padding: 16px;
}

/* Tablet – 768px and wider */
@media (min-width: 768px) {
  .card-grid {
    flex-direction: row;
    flex-wrap: wrap;
  }

  .card-grid .card {
    flex: 1 1 calc(50% - 8px);
  }
}

/* Desktop – 1024px and wider */
@media (min-width: 1024px) {
  .card-grid .card {
    flex: 1 1 calc(33% - 12px);
  }
}
```




Putting It Together — Card Component

```
:root {
  --red: #c8102e;
  --green: #007a33;
}

.card {
  background: white;
  border-radius: 12px;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.1);
  overflow: hidden;
  transition: transform 0.2s ease;
}

.card:hover {
  transform: translateY(-4px);
}

.card__body {
  padding: 20px;
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.card__title {
  color: var(--red);
  font-size: 1.2em;
  font-weight: 700;
}

.card__tag {
  background: var(--green);
  color: white;
  padding: 4px 12px;
```

✓ Key Takeaways

Today you learned:

- 📌 **Selectors** — targeting elements by tag, class, ID
- ⚖️ **Specificity** — how CSS resolves conflicts
- 📦 **Box Model** — margin, border, padding, content
- 🎨 **Variables** — `--name: value` and `var(--name)`
- 💪 **Flexbox** — `display: flex` for 1D component layouts
- 🗂️ **CSS Grid** — `display: grid` for 2D page layouts
- 📱 **Media Queries** — `@media (min-width: ...)` for responsive design



Homework — Profile Page

Build a personal profile card using all today's concepts:

1. ✓ Use an **external** `.css` file
2. ✓ Define **CSS variables** for colours & fonts
3. ✓ Apply **box model** (padding, border, margin)
4. ✓ Use **Flexbox** for layout
5. ✓ Add **hover effects** with `transition`
6. ✓ Make it **responsive** with one media query

```
project/  
├── index.html  
└── style.css
```

Submit via **CodePen** or **GitHub Pages** by next week.



Questions?

Resources:

-  **MDN Web Docs** — developer.mozilla.org
-  **Flexbox Froggy** — flexboxfroggy.com
-  **CSS Tricks** — css-tricks.com
-  **CodePen** — codepen.io



